



FinTrack

Personal Finance Tracker

Rapport de Projet — Juin 2026

Tal Zerbib

Étudiant	Tal Zerbib
Application	FinTrack – Gestion Financière Personnelle
GitHub Frontend	github.com/Talzerbib18/finance-tracker-frontend
GitHub Backend	github.com/Talzerbib18/finance-tracker-backend
Frontend	http://localhost:4200 (Angular 17)
Backend API	http://localhost:8080 (Spring Boot 3.2)
Base de données	PostgreSQL 15 — finance_tracker

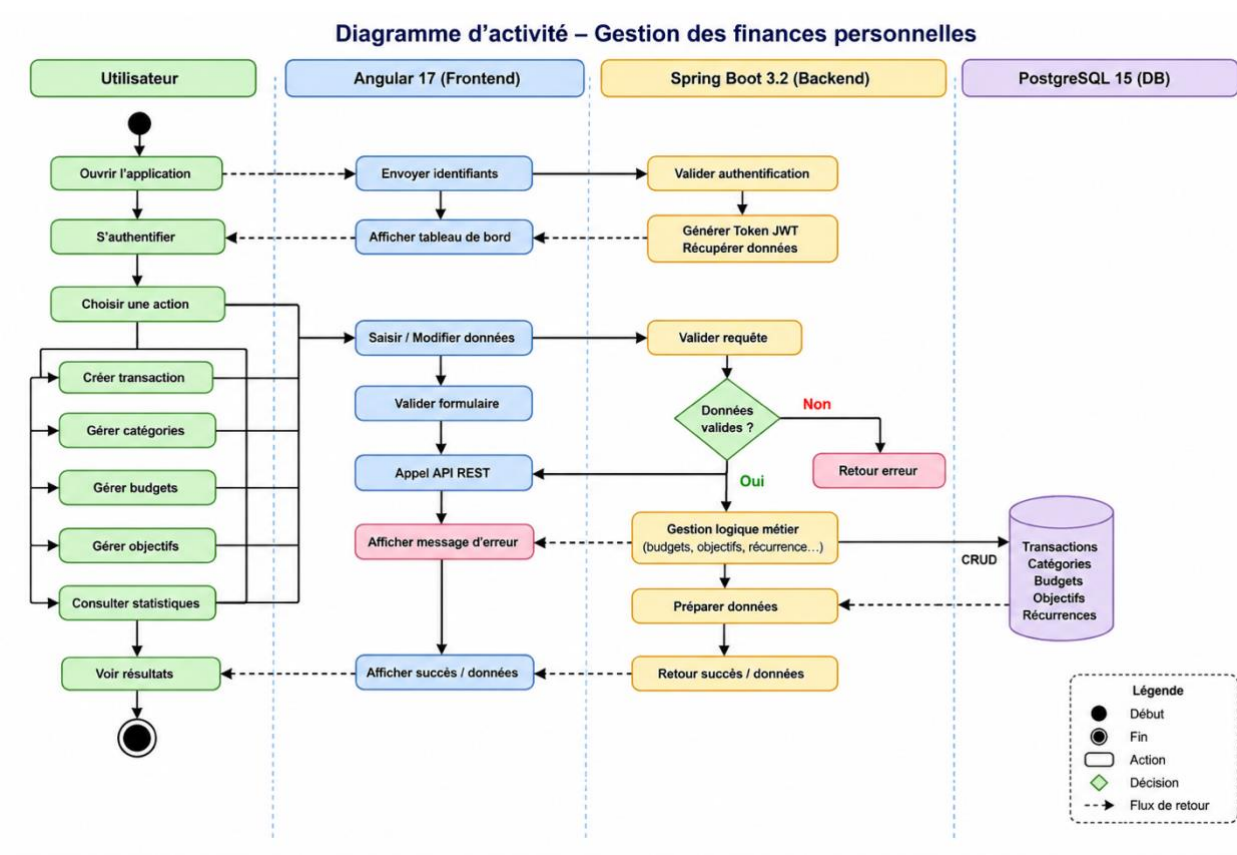
Documentation technique

Choix techniques

Couche	Technologie	Justification
Frontend	Angular 17 + TypeScript	Framework robuste, typage fort, composants standalone
Styles	SCSS + variables CSS	Design system cohérent, dark mode via class toggle
Graphiques	Chart.js 4.5	Librairie légère, compatible Angular
Excel	SheetJS (xlsx)	Import/export .xlsx côté client sans serveur
Backend	Spring Boot 3.2 + Java 21	Productivité, écosystème mature, annotations JPA
ORM	Hibernate + Spring Data JPA	Abstraction BDD, requêtes JPQL typées
BDD	PostgreSQL 15	SGBD relationnel fiable, support LocalDateTime
Build	Maven 3	Gestion des dépendances Java

Diagramme d'architecture

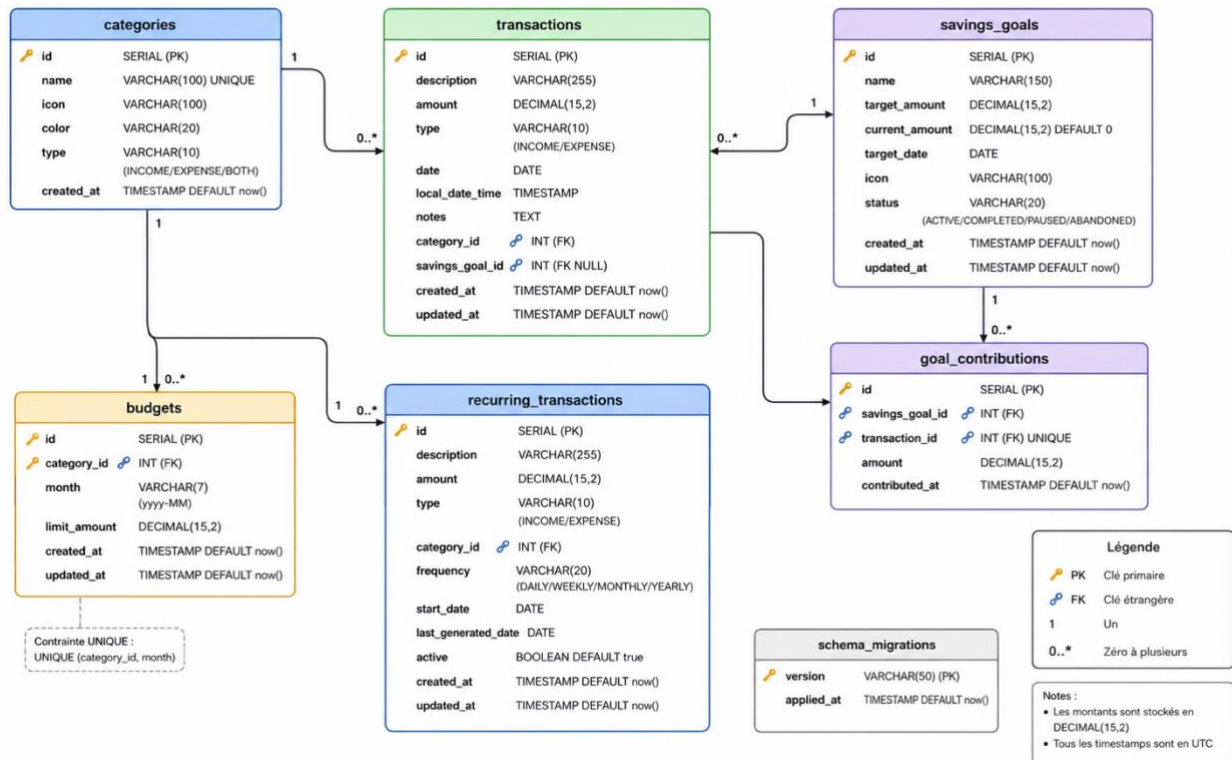
Architecture 3-tiers en couches séparées :



NAVIGATEUR (port 4200)

Diagramme de base de données

Diagramme de base de données – Personal Finance Tracker



Fonctionnalités & API

Pages de l'application

Page	Fonctionnalités clés
Tableau de bord	KPIs période, graphique 12 mois, donut catégories, budgets avec alertes colorées
Transactions	CRUD, 7 filtres, tri, pagination, import/export Excel, validation budget
Objectifs	Contributions atomiques, abandon/réactivation, transactions liées
Catégories	CRUD avec color picker et icon picker emoji
Statistiques	KPIs, graphique 3 lignes, top 6 catégories
Navbar	Dark mode toggle persisté en localStorage

API REST — Endpoints principaux

Méthode	Endpoint	Description
GET/POST/PUT/DELETE	/api/transactions	CRUD + filtres (type, catégorie, goalId, dates, montants, search)
GET/POST/PUT/DELETE	/api/categories	CRUD catégories
GET/POST/PUT/DELETE	/api/budgets	CRUD budgets — GET?month=yyyy-MM avec spentAmount calculé
POST	/api/goals/{id}/contribute	Contribution @Transactional + transaction EXPENSE créée
POST	/api/goals/{id}/abandon	Abandon + transaction INCOME de remboursement
GET	/api/dashboard	Agrégats (startDate, endDate optionnels) + trends
GET	/api/stats/trends	Comparaison période courante vs équivalente précédente
GET	/api/recurring	Templates récurrents (scheduler @01h00 chaque nuit)

Propriétés de la base de données

Propriété	Valeur
SGBD	PostgreSQL 15+
Base	finance_tracker
Port	5432 (défaut)
DDL auto	update — Hibernate gère la création/mise à jour du schéma
Type dates	LocalDateTime (horodatage complet avec heure et minute)
Initialisation	DataInitializer : 15 catégories créées au démarrage si base vide
Contraintes	UNIQUE(category_id, month) sur budgets — nom unique sur categories

Logique métier

Points clés implémentés

- Budget : validation double (frontend + @Transactional backend) → HTTP 400 BudgetExceededException si dépassé
- Contribution objectif : crée simultanément la contribution ET une transaction EXPENSE liée
- Suppression transaction liée à un objectif : transaction de remboursement inverse créée automatiquement
- Abandon objectif : currentAmount remis à 0 + transaction INCOME de récupération
- Scheduler Spring (@Scheduled cron="0 0 1 * * *") : génère chaque nuit les transactions récurrentes
- FlexibleLocalDateTimeDeserializer : accepte yyyy-MM-dd'T'HH:mm, yyyy-MM-dd'T'HH:mm:ss, yyyy-MM-dd
- Filtre client + backend sur les transactions (sécurité timezone pour les dates)

Gestion des budgets

Avant chaque création ou modification de transaction EXPENSE, le système vérifie si le budget mensuel de la catégorie serait dépassé. Si oui, une BudgetExceededException est levée avec les détails (montant dépassé, disponible restant). Cette validation est double : côté frontend (UX immédiate) et backend (sécurité).

Objectifs d'épargne

Chaque contribution à un objectif crée atomiquement une transaction EXPENSE catégorie "Épargne" liée à l'objectif (@Transactional). L'abandon d'un objectif crée une transaction INCOME de remboursement. La suppression d'une transaction liée crée une transaction de remboursement et met à jour le montant de l'objectif.

Transactions récurrentes

Un scheduler Spring (@Scheduled, cron = "0 0 1 * * *") tourne chaque nuit à 01h00 et génère automatiquement les transactions depuis les templates récurrents actifs selon leur fréquence (DAILY, WEEKLY, MONTHLY).

Dashboard dynamique

Le dashboard accepte startDate et endDate. Les totaux, graphiques, catégories et transactions récentes sont calculés sur la période. Un graphique d'évolution compare jusqu'à 6 mois. Des Trends comparent la période sélectionnée à la période équivalente précédente.

Résumé des fonctionnalités

Fonctionnalité	Frontend	Backend
CRUD Transactions + 7 filtres	✓	✓
Tri, Pagination, Duplication	✓	✓
Import / Export Excel & CSV	✓	✓
Budgets mensuels + blocage	✓	✓
Objectifs + contributions atomiques	✓	✓
Abandon / Réactivation + remboursement	✓	✓
Transactions liées aux objectifs	✓	✓
Dashboard avec périodes + trends	✓	✓
Statistiques avancées	✓	✓
Transactions récurrentes (scheduler)	✓	✓
Dark mode persisté	✓	—

Feedback sur le cours

Ce que j'ai aimé

- La liberté de choisir le sujet du projet — cela donne envie de s'impliquer davantage dans ce que l'on construit.
- L'approche full-stack, qui permet de voir comment les deux parties (frontend et backend) communiquent réellement.
- La mise en pratique directe des concepts : REST API, JPA, Angular — apprendre en faisant est bien plus efficace que d'apprendre uniquement en théorie.
- La découverte de Spring Boot, un framework très puissant avec peu de configuration pour des résultats professionnels.

Ce que j'ai appris

- Concevoir et structurer une application web full-stack de A à Z.
- Implémenter une API REST complète avec Spring Boot : entités, repositories, services, DTOs, gestion des exceptions.
- Développer une SPA Angular avec des composants standalone, des services HTTP, et une gestion d'état locale.
- Gérer des relations entre entités (ManyToOne) et des contraintes métier complexes (budget, objectifs, transactions liées).
- Utiliser Git, GitHub, et déployer un projet sur un dépôt public.
- Intégrer des librairies tierces : Chart.js pour les graphiques, SheetJS pour l'import/export Excel.

Ce que j'ai moins aimé

- La configuration initiale de l'environnement (PostgreSQL, Java 21, Maven, Node.js) peut prendre du temps et générer des erreurs difficiles à diagnostiquer pour un débutant.
- La documentation officielle de certaines librairies (notamment JPA avec Hibernate) peut être dense et complexe à appréhender rapidement.

Conclusion

Ce projet m'a permis de développer une application complète et fonctionnelle, représentative de ce que l'on trouve en entreprise. La combinaison Angular + Spring Boot est une stack très demandée sur le marché, et avoir pu la pratiquer sur un projet concret est une vraie valeur ajoutée. Le cours était bien structuré et m'a donné les bases solides pour continuer à progresser en développement web full-stack.